Video Reconstruction

Project proposal

Version 0.3 | 1 November 2026 | English edition

1 Purpose

Develop a production system that analyses a reference video and reconstructs its camera, object layout, motion and timing as an editable 3D scene. A video generation model then uses this structure to produce the final appearance. The system compares the output with the reference and revises the reconstruction or generation settings.

The first question is whether an editable 3D representation enables more precise changes than instructing a video generation model directly. Later stages will extend the workflow across production applications and reuse verified production examples for scene creation from natural language.

2 Problem

Rebuilding a finished video requires a creator to infer how composition, motion, geometry, assets and compositing work together. Generative editing can also alter parts of a shot that the creator wanted to preserve. This project keeps the elements that need control in an editable scene and tests whether those edits remain visible in the generated output.

3 Example use

A creator imports a short clip, reconstructs the scene, and changes only the camera path, replaces the main object, or delays an action. After generative rendering, the creator checks both the requested change and the preservation of unrelated layout and motion. The initial users are individual creators working on short CG shots and motion graphics.

4 Target outcome

The long-term goal is high reference fidelity combined with editability. The first prototype will cover reconstruction, editing, generative rendering and comparison for a limited class of short shots. Pixel-perfect reproduction is not an initial acceptance criterion.

Deliverables include the finished video, editable scene, production specification, generation settings, revision history and evaluation results.

Workflow and initial scope

5 Production workflow

Analyse the input

Split the video into shots and estimate the main objects, depth ordering, camera, motion and event times. Record unobservable scale and hidden geometry as assumptions.

Plan production

Choose what to represent explicitly in 3D and what to leave to generation, based on the intended edits. Specify the tools, inputs, outputs, dependencies and computation budget.

Build an editable scene

Use the Model Context Protocol (MCP) or application APIs to construct geometry, camera and animation. Export depth, edges, object regions or other control passes where supported by the selected model.

Generate the final appearance

Use video-to-video generation (V2V), structural controls and appearance references to produce the final video. Evaluate existing control-capable models first. Consider task-specific fine-tuning if their adherence is insufficient.

Compare and revise

Compare aligned times in the reference and output. Identify whether a mismatch comes from structure, camera, motion or appearance. Revise within the budget and save when the stopping criteria are reached.

6 Initial prototype

Item	Proposed scope
Input	Single shots of 3 to 5 seconds, with 1 to 3 main rigid objects. Focus on translation, rotation and simple contact.
Environment	Blender and one control-capable generation model. Real-time operation is not required.
Edits	Change the camera path, replace the main object, or change an action start time. Change one item per test.
Later stages	Complex human performance, fluids, fracture, long videos, multiple shots, and connections to Houdini and After Effects.

Evaluation and continuation criteria

7 Hypothesis

Adding editable 3D structure and verification will improve edit accuracy and reduce unintended changes and human correction time compared with direct generative editing. This is an untested hypothesis. The pilot will determine whether further development is justified.

8 Comparison conditions

Condition	Purpose
A Direct generation	Provide the reference video and edit instruction directly to a compatible video editing or V2V model.
B LLM and MCP	Use a general-purpose LLM and Blender MCP to reconstruct the video, followed by the shared generative rendering stage.
C Proposed system	Add specialist analysis, structured scene data and managed comparison and revision to condition B.
Ablation of C	After finding an improvement, remove individual features such as tracking or camera estimation to measure their contribution.

Use the same LLM and rendering model for B and C. Prefer the same rendering model for A where feasible; report different models or input modes as separate conditions. Record computation cost and human intervention. Report both equal-budget results and quality versus cost.

9 Test material and measurements

The proposed pilot uses 12 clips: six for development and six held out from implementation and parameter tuning. Keep derivatives of the same source together. Where feasible, run each condition three times and include failed runs. Treat this as an exploratory study, not evidence of general superiority.

Measure edit success, screen-space trajectory and event-time error, preservation of unrelated content, visual fidelity, execution cost and human correction time. Define legitimate consequences of an edit, such as changed occlusion, before scoring preservation.

10 Proposed continuation criteria

Continue if held-out edit success exceeds both baselines and median human correction time is at least 20% lower than the best baseline. Set visual-quality tolerances and cost limits using development clips, then freeze them before evaluation. The 20% threshold is a provisional target, not a result or a claim of statistical significance. If unmet, revise the scope or features before large-scale training.

Replaceable modules and production data

11 Custom system components

Build the shared scene representation, dependency management, input and output adapters, execution history, and comparison and revision controls. Treat the core LLM, analysis software, generation models, application connectors and other plugins as independent modules that can be added, removed, disabled, updated or replaced after deployment.

Replaceable component	Information retained by the system
Core LLM such as GPT or Claude	Specification, plan, constraints and tool results. Do not depend on model-specific internal reasoning.
Analysis OSS and custom plugins	Timestamps, object IDs, coordinate systems, units, camera, estimates, confidence and provenance.
Application connectors and plugins	Scene and asset IDs, changes, export formats, and inputs and outputs of each stage.
Video generation model	Control passes, references, descriptions, seed, model version, generation settings and results.

Alongside open-source software (OSS), develop custom plugins where needed, for example camera correction, motion constraints or output validation. Connect existing and custom functions through shared input and output specifications so each can be independently added, removed, updated or replaced.

Track supported formats, versions and dependencies per module. Select implementations through configuration. Check dependent stages when replacing or removing a module, and use an alternative where needed. Re-evaluate compatibility and quality after changes. Keep application state separate from LLM decisions so saved scenes and history can be retained.

12 From reconstruction to text-driven creation

For each project, link its natural-language specification, scene structure, code, tool observations and actions, intermediate outputs, human corrections, final artifacts, evaluation, environment and asset usage terms. Distinguish executable examples from examples verified for fidelity and editability.

Start with retrieval and reuse of production components. Consider fine-tuning planning or tool use after verified examples accumulate. Test text-only creation on unseen instructions without the project-specific reference video or extracted frames.

These records capture structural production and generative finishing. Details added by the video model do not become explicit 3D authoring steps. Training a dedicated renderer requires a separate set of aligned control inputs and target videos.

Development stages and risks

13 Staged development

Stage	Output	Condition for proceeding
1 Control test	Manually authored 3D scene connected to a generator; before-and-after edit comparison.	If control fails even with a manual scene, revise the rendering approach before automating reconstruction.
2 Build scene	Blender scene construction, shared data, saving and resuming.	Composition and motion can be edited and the workflow can be rerun.
3 Features and evaluation	Specialist analysis, baselines A and B, and held-out results.	Assess improvement against section 10.
4 More applications	Add a Houdini or After Effects connector.	A specific limitation of Blender alone justifies the addition, with a measurable benefit.
5 Reuse and learning	Example retrieval, reusable skills and optional fine-tuning.	Improvement on unseen instructions justifies the cost.

Stages 1 to 3 form the initial project. Set the schedule and budget after confirming available GPU resources, APIs and working hours. Measure stage 1 before purchasing hardware or training a dedicated model. Record runtime, GPU usage, API costs and intervention time throughout.

14 Risks and responses

If generation changes the intended structure, revise the control conditions or model and narrow the scope. Preserve exact text and geometry with conventional rendering and compositing where needed. Expose ambiguous geometry or camera estimates as alternatives and assumptions that a creator can correct.

Remove auxiliary features when improved LLMs or MCP tooling make them unnecessary. Measure their remaining benefit instead of maximising the number of integrations. Pin OSS versions and validate interfaces. Record source and usage terms for training and distributed assets; separate examples with unverified terms.

15 Initial deliverables

Provide a reconstruction-and-editing demo, reproducible code, editable scenes, module interface documentation, evaluation and ablation results, failure cases, and cost and labour records. A technical report should explain what the custom components improved and where they failed.

Prior work and intended contribution

16 Relevant foundations

CineRecon evaluates agents that reconstruct reference videos as editable dynamic 3D scenes. Its published workflow includes Blender MCP, rendering, comparison and revision. [1] Video to Blender supports editable production through Geometry Nodes, Shader Nodes and compositor graphs. [2]

SceneCraft combines scene graphs, Blender code, rendered-image feedback and reuse of common production functions. [3] Matrix-Game 4.0 separates game-state execution from a trained renderer that converts simplified scene renderings into detailed video. [4] Wan Control variants provide public models with conditioning such as depth and pose. [5]

17 Contribution to test

Connecting existing models and production applications is not itself the proposed novelty. The project combines video-to-structure reconstruction with structure-conditioned generation and measures edit accuracy, preservation of unrelated content and human correction time. Academic novelty remains subject to a detailed literature review and experiments.

First test the benefit of the structural representation. Expand application integration and data collection only where useful effects are established, rather than attempting general video reconstruction and dedicated renderer training together.

References

[1] CineRecon Reconstruction benchmark and generation workflow.
https://github.com/Unc1eW4ng/CineRecon

[2] Video to Blender Agent skill and editable node library.
https://github.com/gokayfem/video-to-blender

[3] SceneCraft ICML 2024 Blender code generation and library reuse.
https://proceedings.mlr.press/v235/hu24g.html

[4] Matrix Game 4.0 Separate state execution and learned rendering.
https://arxiv.org/abs/2610.27999

[5] Wan2.2 Fun Control Control-capable models and implementation.
https://github.com/aigc-apps/VideoX-Fun

Sources accessed 1 November 2026. Prior-work descriptions are based on public documentation. System performance, training requirements, budget and commercial viability remain to be tested.